

Refined Product Backlog v2

(Added Acceptance Criteria)

Priority 1: High (Foundational & Sprint 2 Candidates)

These items must be tackled first as they establish the app's environment, security, device context, and basic navigation.

1. Device Identity and Site Binding (8 SPs, < 2 sprints)

- **(8 SPs) User Story (Device Identity and Site Binding):** As the MoPS application, I want each continuity record attributable to a known device and site so that operations remain traceable and auditable.
- **Tasks:**
 - Register/configure a unique device ID and validate its status on app startup.
 - (1-2 hrs, < 1 sprints)
 - Bind the device to an assigned site.
 - (1-2 hrs, < 1 sprints)
 - Display the device ID in diagnostics/settings.
 - (1-2 hrs, < 1 sprints)
 - Ensure device ID, site ID, and lane context are included in all continuity records, incidents, and manual gate logs.
 - (1-2 hrs, < 1 sprints)
 - Write device/site attribution tests.
 - (1-2 hrs, < 1 sprints)

- **Acceptance Criteria:**

- Every continuity record contains the correct device ID and site ID.
- Device identity is displayed in Diagnostics/Settings.
- Device status is validated during app startup.
- Records cannot be submitted without valid device/site attribution.

2. Authentication and Access Control (26 SPs, > 3 sprints)

- **(13 SPs points) User Story 1 (Operator Login):** As a parking operator, I want to securely authenticate into an assigned MoPS device so that continuity operations can be performed only by authorized personnel within the correct site context.
- **(13 SPs points) User Story 2 (Role-Based Access):** As an authorized user, I want app actions restricted by role so that continuity operations remain controlled.
- **Tasks:**
 - Design login screen UI and implement username/password form with loading/error states.
 - (3-5 hrs, < 2 sprints)
 - Connect login API, store session tokens securely, and implement logout.
 - (3-5 hrs, < 2 sprints)
 - Crucial: Validate assigned site context at login/startup; deny or warn on unrecognized device or site mismatch. Retrieve and attach device ID post-login.
 - (3-7 hrs, > 2 sprints)
 - Define user roles, hide restricted UI actions, and restrict high-risk actions (manual gate logging, overrides).
 - (3-5 hrs, < 2 sprints)
 - Capture user role in action metadata for audits.
 - (3-5 hrs, < 2 sprints)

- **Acceptance Criteria:**

- Users can log in using valid credentials.
- Invalid credentials display an appropriate error message without exposing sensitive information.
- Session token is securely stored after successful login.
- Device ID and assigned site are validated during login/startup.
- Unrecognized devices or site mismatches prevent or warn the user before continuing.
- Users can securely log out and invalidate the session.
- Only authorized roles can access restricted screens and actions.
- Restricted functions are hidden or disabled for unauthorized users.
- High-risk actions (manual gate logs, override requests) require appropriate roles.
- User role is included in submitted action metadata.

3. Frontend UI/UX: Operator Dashboard (8 SPs, < 2 sprints)

- **(8 SPs) User Story (Operator Dashboard):** As an operator, I want a clear dashboard so that I can quickly perform continuity actions during operational disruptions.
- **Tasks:**
 - Design responsive dashboard (optimize for tablet/mobile).
 - (1-2 hrs, < 1 sprint)
 - Add quick-access buttons and establish navigation flow.
 - (1-2 hrs, < 1 sprint)
 - Display: current device ID, site context, connectivity status, unsynced record count, and a clear continuity-mode state or warning banner.
 - (3-5 hrs, < 2 sprints)

- **Acceptance Criteria:**

- Dashboard provides quick access to major continuity functions.
- Current device ID and assigned site are visible.
- Connectivity status is displayed.
- Unsynced record count is displayed.
- Continuity mode banner/warning appears when appropriate.
- Interface is usable on supported mobile/tablet devices.

4. Backend Integration and Core Setup (21 SPs, > 3 sprints)

- **(21 SPs) User Story (API Integration):** As for the MoPS application, I want to communicate with the specific ExitPass backend services required for continuity workflows so that continuity records can be synchronized correctly.
- **Tasks:**
 - Configure API client with authentication headers, retry handling, and timeout handling.
 - (3-7 hrs, > 2 sprints)
 - Integrate required `/v1/ops/*` endpoints only (Handle canonical error responses).
 - (3-7 hrs, > 2 sprints)
 - Attach correlation ID, site ID, and device ID where required.
 - (3-5 hrs, < 2 sprints)
 - Add API contract tests for continuity endpoints.
 - (3-5 hrs, < 2 sprints)

- **Acceptance Criteria:**

- App communicates only with approved `/v1/ops/*` endpoints.
- Authentication headers are included in every request.
- Correlation ID, site ID, and device ID are attached where required.
- Retry and timeout handling work correctly.
- API contract tests pass.

Priority 2: Medium (Core Continuity Workflows)

These items form the primary functional loop of the app. They should be refined next, potentially splitting across Sprint 2 and beyond.

5. Continuity Transaction Capture (30 points, < 3 sprints)

- **(15 points) User Story 1 (Create Continuity Transaction)** : As an operator, I want to capture continuity-mode payment-related transaction evidence during service disruptions so that operations may continue while preserving records for later reconciliation and review.
- **(15 points) User Story 2 (Continuity Transaction History)** : As an operator, I want to review previously captured continuity records and their processing status so that I can verify operational evidence and synchronization outcomes.
- **Tasks:**
 - Design transaction form with validation (capture timestamps, operator ID, site ID, device ID, lane context).
 - (8 points, < 2 sprints)
 - Consume the provided mock Payment Orchestrator interface for continuity workflow simulation.
 - (8 points, < 2 sprints)
 - Save locally, map local record state to backend MoPS lifecycle state, and submit to backend (prevent duplicates via idempotency).
 - (7 points, < 2 sprints)
 - Create transaction history screen with filtering, showing local submission state vs. backend reconciliation status/exception flags.
 - (7 points, < 2 sprints)

- **Acceptance Criteria:**

- The operator can create a continuity transaction with all required fields.
- Transaction is validated before saving.
- Timestamp, operator ID, device ID, site ID, and lane are captured.
- The record is saved locally when offline.
- Duplicate submissions are prevented.
- Sync status and backend lifecycle status are displayed separately.
- Users can view previously recorded continuity transactions.
- Search and filtering work correctly.
- Transaction details include evidence, notes, timestamps, and operators.
- Local sync state and backend reconciliation status are clearly displayed.

6. Offline Sync and Retry (13 SPs, < 3 sprints)

- **(13 SPs) User Story (Offline Sync and Retry):** As an operator, I want transactions to sync automatically once connectivity returns so that records are not lost during outages.
- **Tasks:**
 - Design offline queue and store unsynced records locally.
 - (3-5 hrs, < 2 sprints)
 - Separate local queued state from backend accepted state.
 - (3-5 hrs, < 2 sprints)
 - Detect connectivity restoration and retry failed uploads (prevent duplicate resubmission using idempotency key).
 - (3-5 hrs, < 2 sprints)
 - Retry evidence upload alongside record upload where needed.
 - (3-5 hrs, < 2 sprints)

- **Acceptance Criteria:**

- Transactions are stored locally while offline.
- The app detects restored connectivity automatically.
- Unsynced records upload automatically.
- Sync conflicts are handled appropriately.
- Duplicate uploads are prevented using idempotency keys.
- Evidence uploads retry together with their records.

7. Operational Evidence Capture (20 points, < 3 sprints)

- **(20 points) User Story (Evidence Attachment):** As an operator, I want to attach supporting evidence to continuity records so that reconciliation and audit activities have sufficient operational context.
- **Tasks:**
 - Implement controlled photo capture/upload and an operational notes field.
 - (5 points, < 2 sprints)
 - Compress images, validate format, and tag evidence type.
 - (5 points, < 2 sprints)
 - Prevent uncontrolled permanent storage on the device where possible.
 - (5 points, < 2 sprints)
 - Sync notes and evidence with the transaction record, preserving author/timestamp metadata.
 - (5 points, < 2 sprints)

- **Acceptance Criteria:**

- Users can attach supported evidence (photo/upload).
- Invalid formats are rejected.
- Images are compressed before storage/upload.
- Evidence includes timestamps and metadata.
- Failed uploads can be retried.
- Evidence is linked to its associated transaction.

Priority 3: Low (Edge Cases, Exceptions, & Handoff)

These items handle the anomalies and the final data pipeline. They are candidates for later sprints.

8. Incident & Exception Management (15 points, < 2 sprints)

- **(7 points) User Story 1 (Incident Logging):** As an operator, I want to record operational incidents associated with continuity operations so that disruptions, exceptions, and operational risks remain traceable.
- **(8 points) User Story 2 (Exception Tagging):** As an operator, I want to classify continuity records using approved exception categories so that reconciliation personnel can review exceptional cases consistently.
- **Tasks:**
 - Create incident form using controlled incident categories and attach evidence.
 - (3 points, < 1 sprint)
 - Capture device ID/site ID and link the incident to a transaction when applicable.
 - (4 points, < 1 sprint)
 - Implement tagging UI using structured reason codes (do not invent ad hoc statuses).
 - (4 points, < 1 sprint)
 - Submit exception context for backend reconciliation and display backend state if returned.
 - (4 points, < 1 sprint)
- **Acceptance Criteria:**
 - Users can log incidents using predefined categories.

- Device ID, site ID, timestamp, and evidence are captured.
- Incident can be linked to an existing transaction.
- Incident history is viewable after submission.
- Users can assign predefined exception categories or reason codes.
- Free-form exception statuses are not allowed.
- Exception information is submitted with the continuity record.
- Backend reconciliation status is displayed when available.

9. Manual Operation Support (15 points, < 2 sprints)

- **(8 points) User Story (Manual Gate Action Logging):** As an operator, I want to document manual gate operations performed during continuity scenarios so that all exceptional actions remain attributable and auditable.
- **(7 points) User Story (Override Request Workflow):** As an operator, I want to submit override requests containing justification and supporting context so that exceptional situations can be escalated through authorized backend workflows.
- **Tasks:**
 - Create forms to capture the reason for manual actions/overrides (require justification text or reason code).
 - (4 points, < 1 sprint)
 - Capture operator identity, timestamps, device identity, and site/lane context.
 - (4 points, < 1 sprint)
 - Link to incident records or parking sessions where known.
 - (3 points, < 1 sprint)
 - Submit requests, prevent duplicates, and display read-only acknowledgment or approval outcomes returned by the backend.
 - (4 points, < 1 sprint)

- **Acceptance Criteria:**

- Users can log manual gate actions with a required reason.
- Operator identity, timestamp, device ID, site ID, and lane are recorded.
- Duplicate manual logs are prevented.
- Manual logs can be linked to incidents or parking sessions when available.
- Submission success/failure is clearly displayed.
- Users can submit an override request with required justification.
- Request status is displayed.
- Read-only approval outcome is shown if returned by the backend.
- No supervisor approval interface exists within the app.

10. Reconciliation Handoff & Audit Traceability (10 points, < 2 sprints)

- **(10 points) User Story (Reconciliation Submission and Audits):** As for the MoPS application, I want to integrate only with approved ExitPass continuity-service endpoints so that continuity records can be transmitted securely and consistently.
- **Tasks:**
 - Implement MoPS continuity-record upload/import trigger.
 - (3 points, < 1 sprint)
 - Show whether a record is pending reconciliation, disputed, rejected, or reconciled.
 - (3 points, < 1 sprint)
 - Ensure all app actions submit required audit metadata (timestamps, operator, device, site, lane) through *normal* backend workflows (do not use direct audit endpoints).
 - (4 points, < 1 sprint)

- **Acceptance Criteria:**

- Continuity records can be submitted for reconciliation.
- Acknowledgment is received and displayed.
- Reconciliation status (Pending, Reconciled, Rejected, Disputed) is shown when available.
- Failures are communicated clearly to the user.